

SENAI · M1S10 · Semana 10

HTML & CSS profissional

Classes · Herança e cascata · Predominância · O mapa da página

Roteiro da aula

A. Classes CSS

O que cada uma faz: container, layout, componente, utilitária, estado.

B. Herança e cascata

Quem recebe de quem. Quem vence quando duas regras brigam.

C. Predominância

O placar de especificidade e como resolver conflitos.

D. O mapa do HTML

Div, main, section, article. Qual classe começa. Padrão profissional.

Regra de ouro: HTML diz *o que a coisa é*. CSS diz *como ela parece*. A classe é a ponte.

TRILHA A

Classes CSS

Nome, papel e função de cada uma

O que é uma classe

```
<!-- HTML: etiqueta no elemento (sem ponto) -->
<button class="botao">Enviar</button>

/* CSS: a regra procura a etiqueta (com ponto) */
.botao { background:#1b4f72; color:#fff; padding:10px 20px; }
```

- Pode ter **várias classes** no mesmo elemento: `class="botao botao--grande"`
- Nome por **função**, não aparência: `botao-primario`, não `botao-azul`
- Minúsculas com hífen: `card-produto`

Estilize com **classe**. Reserve o `id` para âncora e JavaScript.

Os 5 papéis de uma classe

Papel	Pergunta que responde	Exemplos
Estrutura	Largura e ritmo da página?	<code>container</code> <code>secao</code>
Layout	Como os filhos se organizam?	<code>flex</code> <code>grid</code> <code>row</code>
Componente	Qual peça reutilizável?	<code>botao</code> <code>card</code> <code>navbar</code>
Utilitária	Qual ajuste único?	<code>mt-3</code> <code>text-center</code> <code>hidden</code>
Estado / modificador	Qual variação ou situação?	<code>is-open</code> <code>botao--grande</code>

container: o padrão "faixa + container"

```
.container {  
  max-width: 1100px; /* teto, nunca estoura o celular */  
  margin: 0 auto;    /* centraliza */  
  padding: 0 16px;  
}
```

```
<section class="secao">           <!-- faixa: fundo 100% -->  
  <div class="container">         <!-- conteúdo: largura limitada -->  
    ...  
  </div>  
</section>
```

Por que `max-width` e não `width` ? Largura fixa estoura o celular.

Layout: flex ou grid?

Flexbox: uma dimensão

Menu, barra de botões, centralizar, linha de cards.

```
.flex { display:flex; gap:16px; }
```

Grid: duas dimensões

Galeria, catálogo, layout de página.

```
.grid { display:grid; gap:16px;  
  grid-template-columns:  
    repeat(auto-fit,minmax(240px,1fr)); }
```

A classe de layout vai no **pai**. Quem se organiza são os **filhos diretos**.

Componente, utilitária e estado

Componente

Entrega a **peça inteira**.

`.card` `.botao`

Utilitária

Faz **uma coisa só**, em qualquer lugar.

`.mt-3` `.hidden`

Estado

Situação que **muda** com o uso (JS liga/desliga).

`.is-open` `.ativo`

```
<button class="botao botao--primario botao--grande mt-3">  
      componente  variação          variação    utilitária
```

BEM: `card` (bloco) · `card__titulo` (parte) · `card--destaque` (variação)

Dicionário comercial

Quero...	CSS puro	Bootstrap	Tailwind
Centralizar a largura	<code>container</code>	<code>container</code>	<code>mx-auto max-w-6xl</code>
Ligar flex	<code>flex</code>	<code>d-flex</code>	<code>flex</code>
Metade da largura	<code>coluna-metade</code>	<code>col-md-6</code>	<code>md:w-1/2</code>
Botão principal	<code>botao botao--primario</code>	<code>btn btn-primary</code>	<code>bg-blue-700 px-4 py-2</code>
Esconder	<code>hidden</code>	<code>d-none</code>	<code>hidden</code>

Quem entende a base lê qualquer framework: o `row` do Bootstrap é flex + wrap por baixo.

TRILHA B

Herança e efeito cascata

De quem o elemento recebe? Quem vence a briga?

O que é herança

body font-family e color declarados aqui, uma vez

main

section

p herda fonte e cor sem ninguém pedir

- O **filho recebe do pai** certos valores, automaticamente.
- Declaração **direta** no filho sempre vence a herdada.

O que herda e o que não herda

✓ Herdam (texto)

- `color`
- `font-*`
- `line-height`
- `text-align`
- `cursor` , `visibility`

✗ Não herdam (caixa)

- `margin` , `padding`
- `border`
- `background`
- `width` , `height`
- `display` , `position`

Se toda `div` herdasse a borda do pai, a página viraria uma cebola de molduras.

Receita profissional da base

```
body { font-family: Arial, sans-serif;  
      line-height: 1.6; color: #263238; }  
a { color: inherit; } /* link não herda sozinho */  
input, button, textarea, select { font: inherit; }
```

Valor	Faz
<code>inherit</code>	Força herdar do pai
<code>initial</code>	Volta ao padrão do CSS
<code>unset</code>	inherit se herda, initial se não

Efeito cascata: as 3 perguntas, nesta ordem

- 1 **Origem e importância** — navegador < autor? Tem `!important` ?
- 2 **Especificidade** — qual seletor é mais forte?
- 3 **Ordem no código** — empatou? **A última vence.**

```
p      { color: blue; font-size: 18px; }  
.aviso { color: green; }  
/* <p class="aviso"> → verde (da classe) e 18px (do p).  
   Regras se somam; só brigam onde discordam. */
```

Veja a cascata no DevTools

- 1 Aperte **F12**
- 2 Clique no elemento
- 3 Painel **Estilos**

~~Riscada~~ = perdeu a briga.

"Inherited from..." = veio por herança.

Aba **Computed** = valor final.

TRILHA C

Predominância

Quem vence: classe, id ou tag?

O placar tem 4 colunas



Compara da **esquerda para a direita**. A primeira coluna maior ganha; as outras nem são olhadas.

p	0-0-0-1	.card h2	0-0-1-1
.card	0-0-1-0	#lojas .card h2	0-1-1-1

Uma classe vence 7 tags empilhadas

```
.botao { color: red; } /* 0-0-1-0 */  
html body main section div p a { color: blue; } /* 0-0-0-7 */  
/* vermelho: 1 na coluna C ganha de 7 na coluna D */
```

Um id vence 100 classes

#topo (0-1-0-0) > .a.b.c... (0-0-N-0)

Herança não entra no placar

Valor herdado perde para **qualquer** regra direta.

Empate nas 4 colunas? **Vence a última** escrita no CSS.

O link que não muda de cor

```
.menu a { color: white; } /* 0-0-1-1 */  
.destaque { color: gold; } /* 0-0-1-0 → perde! */
```

```
.menu .destaque { color: gold; } /* 0-0-2-0 → vence, sem !important */
```

Armas, da mais saudável à pior

1 Reordenar · 2 Classe simples · 3 Mais uma classe no seletor · 4 `:where()` · 5 `!important` · 6 id / inline

Hierarquia de poder, de propósito

Tags (base) 0-0-0-1 · o piso do site

Classes de componente 0-0-1-0 · a regra de cada peça

Modificadores e estados vêm depois do componente e vencem

Utilitárias por último · a palavra final do dev

Mantra: baixa especificidade + ordem organizada = nunca lutar com a cascata.

Quiz rápido: quem vence?

```
.titulo { color: red; }  
.titulo { color: green; }
```

→ ?

```
#topo { color: red; }  
.a.b.c.d.e.f { color: blue; }
```

→ ?

```
.menu a { color: white; }  
.destaque { color: gold; }
```

→ ?

```
.btn { background:blue !important; }  
<button style="background:red">
```

→ ?

Respostas: verde (última) · vermelho (id) · branco (0-0-1-1 > 0-0-1-0) · azul (!important)

TRILHA D

O mapa do HTML profissional

Da raiz ao avançado: o que vem primeiro, o que vai dentro de quê

Esqueleto de uma página profissional

`<html lang="pt-BR">` ① documento

`<head>` ② invisível: charset · viewport · title · link do CSS

`<body>` ③ o que o usuário vê · fonte base (herança)

`<header>` ④ topo do site

`<main>` ⑤ conteúdo único · só 1

`<footer>` ⑧ base

`section → div.container → h1 · p · .card` ⑥ ⑦ dentro do main

As 5 camadas

Camada	Elementos	Pergunta
1 Documento	html head body	Como o navegador lê?
2 Moldura	header main footer	Quais as 3 grandes áreas?
3 Seções	section article aside	Quais os assuntos?
4 Layout	div.container div.grid	Largura e colunas?
5 Conteúdo	h1 p a img .card .botao	O que se lê e se clica?

Camadas 1–3: **tags semânticas**. Camada 4: reino da **div**. Camada 5: mistura.

Div, main, section ou article?

<main>

Conteúdo principal e único da página. **Só um.**

<section>

Grupo sobre **um tema, com título** (h2).

<article>

Peça **independente**: produto, post, card.

<aside>

Apoio, dispensável: lateral, "leia também".

<nav>

Grupo principal de links.

<div>

Último recurso. Só agrupa para o CSS.

Existe tag com o significado? Use. Senão, `div`.

Os pares que confundem

section ou div?

Tirando o CSS, a caixa ainda faz sentido no documento? **Sim** → **section**. Não → **div**.

section ou article?

Posso copiar para outro site e continua fazendo sentido? **Sim** → **article**.

link ou botão?

Muda de página → `a` . Executa ação → `button` .

Erros clássicos

Dois `main` · "divite" · div com onclick · section só para pintar fundo.

A ordem de escrita do CSS

- 1 Reset
- 2 Variáveis (`:root`)
- 3 Base (tags + herança)
- 4 Estrutura (`.container`)
- 5 Layout (`.grid` , `.flex`)
- 6 Componentes
- 7 Modificadores e estados
- 8 Utilitárias e media queries

Do **geral ao específico**: em empate, o que vem depois vence. A ordem no atributo `class=""` do HTML **não** importa.

Nomenclatura BEM e organização

```
.card          /* Bloco */  
.card__titulo  /* Elemento */  
.card--destaque /* Modificador */
```

```
projeto/  
├─ index.html  
├─ estilo.css  
├─ script.js  
└─ assets/
```

- Todo seletor é **uma classe só** (0-0-1-0): sem briga de peso.
- Prefixo = dono: `hero-titulo`, `card-loja`, `coluna-dados`.
- Um idioma só, sem numeração (`caixa2`).

Os 5 níveis: onde você está?

1 Raiz	A página funciona (tags básicas, classes, cor e fonte)
2 Estrutura	header / main / section / footer, container
3 Layout	Flex, grid, rem, mobile-first
4 Cascata	Calcula especificidade, sem id nem !important
5 Arquitetura	BEM, variáveis, acessibilidade, escolha de biblioteca

Cola de bolso

Qual tag?

Tem tag com o significado? Use.
Senão, div.

Estilo não aparece?

Seletor acha? Alguém mais
específico? Empate na ordem?

Onde a fonte?

No `body`. Os filhos herdam.

Qual classe primeiro?

Base → estrutura → componente
→ modificador → utilitária.

Como nomear?

O que é, não como parece.

Flex ou grid?

Uma dimensão: flex. Duas: grid.

Para casa: abra a página de contato da Bikcraft e localize cada camada do mapa: `header`, `main`,
`section`, `article`, `footer`.